

Unsupervised Deep Learning

Week 1: Introduction to Generative Modelling

October 2026

Magda Gregorová - magda.gregorova@thws.de



Licensed according to CC-BY-SA 4.0

1. Motivation
2. Unsupervised Learning - Generative Modelling
3. Learning a Distribution
4. Course Outline / Logistics

Generative Models: Text and Beyond

Chatbot	Developer
ChatGPT	OpenAI
Claude	Anthropic
Gemini	Google
Le Chat	Mistral
Qwen Chat	Alibaba
DeepSeek	DeepSeek

Domain	Try it
Images	Stable Diffusion
Speech	Kokoro
Audio	Stable Audio Open
3D	TRELLIS.2

Text is only one data type

- **Proteins, drug design:** new molecules to specification
- **Materials:** new crystal structures
- **Weather, climate:** ensembles of forecasts
- **Aerodynamic, mechanical design:** shapes under constraints
- **Medical imaging:** synthetic scans, reconstruction

Same models, different data

What Do These Have in Common?

Text - images - speech - audio - 3D - proteins - materials -
weather

- Output looks real
- Different outcome every time
- Many valid outputs - prompt guides, does not decide

Next: how is real data made? Can we make it ourselves?

Real Data Comes From a Process

Data	Process
Photo	camera, light
Speech	microphone, voice
Text	people writing
Molecule	nature

Every run: different example

Goal: examples without the process - from mathematical model

How to Generate Examples?

- Image $\mathbf{x}$: 512×512 pixels $\times$ 3 colours $\approx$ 800 000 numbers
- Computer draws random numbers: $\mathbf{x} \sim p(\mathbf{x})$ - different outcome every time



Every pixel random



Real photo

Distribution decides what comes out

Which Distribution?

- Real images - very likely
- Noise - very unlikely
- Many likely $\mathbf{x}$ - many valid outputs

By hand: impossible - $\approx 800\,000$ numbers per image

Learn it from real examples: $\mathcal{D} = \{\mathbf{x}_1, \dots, \mathbf{x}_N\}$

Generative modelling: learn $p(\mathbf{x})$ from examples - sample new $\mathbf{x}$

Supervised vs Unsupervised

	Supervised	Unsupervised
Data	pairs $(\mathbf{x}, \mathbf{y})$	examples $\mathbf{x}$ only
Learn	$p(\mathbf{y} \mathbf{x})$	$p(\mathbf{x})$
Output	prediction	new example
Right answer	one per input	many valid outputs

Prompt = label: text-to-image learns $p(\mathbf{x} | \mathbf{y})$ - generative, paired data

What Can We Do With a Learned Distribution?

Generate: new examples - **main focus of this course**

Also possible - end of course, if time allows:

- **Complete:** fill in missing parts - inpainting, super-resolution
- **Evaluate:** how typical is this image? - anomaly detection
- **Structure:** similar images, interpolation, useful features

How Do We Learn a Distribution?

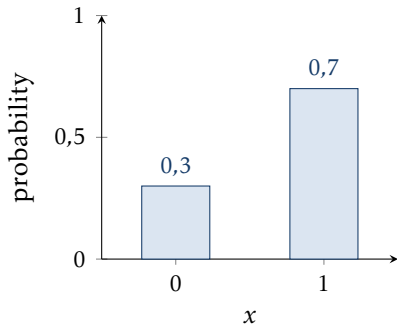
Start simple: one binary variable $x \in \{0, 1\}$ - coin flip, pixel on / off

True generative process: biased coin flip

Samples: 1, 0, 1, 1, 0, 1, 1, 1, 0, 1

Learn: count - $p(x = 1) = 7/10 = 0.7$

Generate: sample $x \sim p(x) = \text{Bernoulli}(0.7)$



Several Categories: Restaurant Orders

Next: one variable with 5 values $x \in \{\text{soup, pizza, pasta, burger, salad}\}$

True generative process: choosing a dish

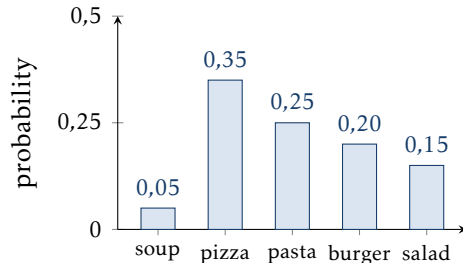
Samples: pizza, pasta, pizza, burger, pizza, salad, ... (20 orders)

Learn: count

$$p(x = \text{soup}) = 1/20 = 0.05$$

$$p(x = \text{pizza}) = 7/20 = 0.35$$

...



Generate: sample $x \sim p(x) = \text{Categorical}(0.05, 0.35, 0.25, 0.20, 0.15)$

Question: how many parameters do we need?

One Continuous Variable: Histogram

Next: one continuous variable - waiting time x for a dish, in minutes

True generative process: preparing a dish

Samples: 3.7, 3.2, 2.9, 3.8, 2.8, ... (50 orders)

Learn: count examples per bin

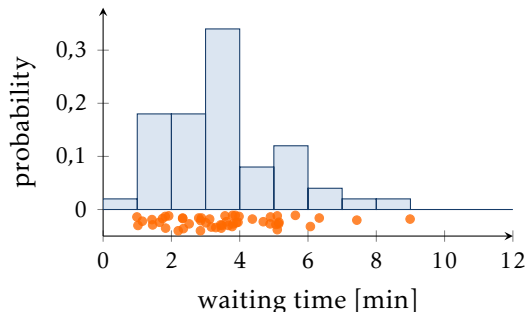
$$p(0 \leq x < 1) = 1/50 = 0.02$$

$$p(3 \leq x < 4) = 17/50 = 0.34$$

...

Generate: sample bin $\sim p(x) = \text{Categorical}(0.02, 0.18, \dots)$, then x inside bin

Question: how many parameters do we need? other issues?



Histogram: Problems

Parameters: one probability per bin - 12 bins, 11 free

Problems:

- Bin width arbitrary - changes the picture
- Jagged, noisy with few examples
- Empty bins - zero probability, outside range too

Idea: assume smooth shape with few parameters

Assume Shape (Family): Gaussian

Model: bell shape with 2 parameters -
mean μ , spread σ , $p(x) = \mathcal{N}(x | \mu, \sigma^2)$

Learn: average and spread of examples

$$\hat{\mu} = \frac{1}{N} \sum_i x_i = 3.6$$

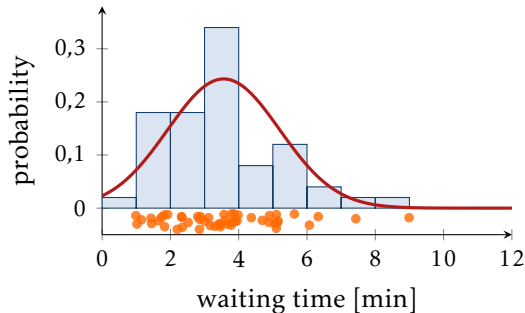
$$\hat{\sigma}^2 = \frac{1}{N} \sum_i (x_i - \hat{\mu})^2, \hat{\sigma} = 1.6$$

Generate: sample

$$x \sim p(x) = \mathcal{N}(x | \hat{\mu} = 3.6, \hat{\sigma}^2 = 1.6^2)$$

Parameters: 2 instead of 11 - but shape assumed

Trade-off: wrong shape - wrong model



Two Groups: One Gaussian Is Not Enough

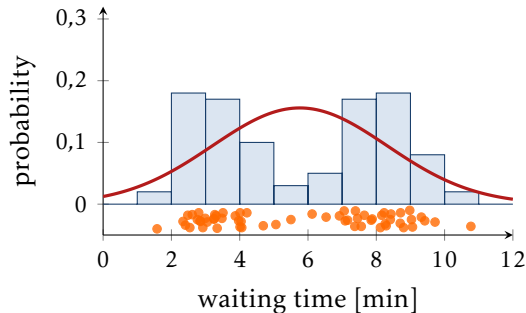
Data: waiting times - two groups

Gaussian fit:

$$p(x) = \mathcal{N}(x \mid \hat{\mu} = 5.8, \hat{\sigma}^2 = 2.6^2)$$

Problem: peak between groups

Idea: combine several Gaussians



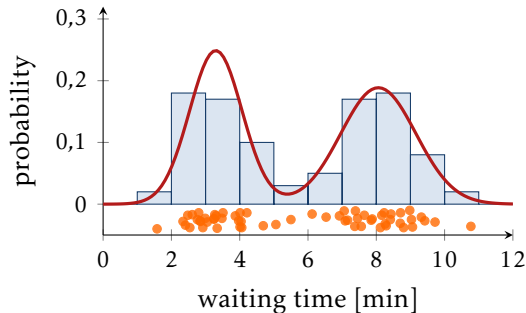
Mixture of Gaussians

Model: 2 bell shapes

$$p(x) = \pi_1 \mathcal{N}(x | \mu_1, \sigma_1^2) + \pi_2 \mathcal{N}(x | \mu_2, \sigma_2^2)$$

Learn: estimate means $\hat{\mu}_1, \hat{\mu}_2$, spreads $\hat{\sigma}_1, \hat{\sigma}_2$, weights $\hat{\pi}_1, \hat{\pi}_2$

Generate: pick group $k \sim Be(\hat{\pi}_1)$, then $x \sim \mathcal{N}(x | \hat{\mu}_k, \hat{\sigma}_k^2)$



Question: how many parameters for 2 Gaussians? 3 Gaussians? k Gaussians?

Two Categorical Variables

Examples: (pizza, beer), (pasta, wine), (burger, beer), (pizza, beer), ...

Idea 1: each pair a category $p(\text{dish}, \text{drink})$

Learn: count pairs, divide by N

Generate: sample pair from $\text{Cat}(15)$

Question: empty cells, $p(\text{soup}, \text{wine}) = 0?$

Idea 2: independent $p(\text{dish})$, $p(\text{drink})$

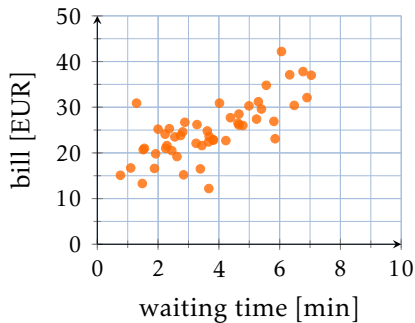
Learn: count individual categories

Generate: sample independently $\text{Cat}(\text{dish})$,
 $\text{Cat}(\text{drink})$

Question: parameters? issues?

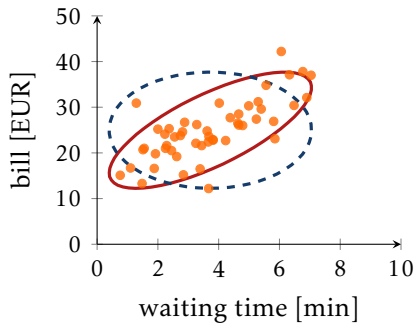
	water	beer	wine	marginal
soup	0.05	0.00	0.00	0.05
pizza	0.05	0.25	0.05	0.35
pasta	0.05	0.00	0.20	0.25
burger	0.05	0.15	0.00	0.20
salad	0.10	0.00	0.05	0.15
marginal	0.30	0.40	0.30	1.00

Two Continuous Variables



Histogram: 100 cells, most empty

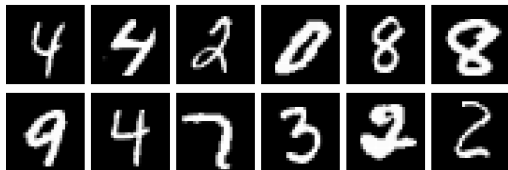
Independent: 10 cells each, ???



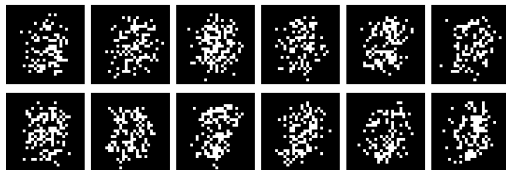
Gaussian: 2 means, 2×2 covariance mat

Independent: 2 means, 2 vars, ???

Images: 784 Variables



Real digits, 28×28 pixels



Independent pixels

- Histogram: $2^{784} \approx 10^{236}$ possible configurations = cells, even for black / white
- Independent pixels: 784 params $\Rightarrow$ speckle - lost dependence = no structure
- One Gaussian: blurry blobs

Many Variables - curse of dimensionality

Variables	Histogram cells	Gaussian parameters
1	10	2
2	10^2	5
3	10^3	9
d	10^d	$d + d(d + 1)/2$
784	10^{784}	308 504

Histogram: cells grow exponentially $>$ atoms in universe ($\approx 10^{80}$) - **most empty!**

Gaussian: parameters grow quadratically - **one bell shape blob!**

Need: flexible model with few parameters $\Rightarrow$ neural networks

- **Generative modelling:** learn $p(\mathbf{x})$ from examples - sample new $\mathbf{x} \sim p(\mathbf{x})$
- **Real data:** output of a process - model replaces the process
- **Simple families:** Bernoulli, categorical, histogram, Gaussian, mixture
- **Few variables:** learn by counting, average and spread
- **Many variables:** cells explode, independence loses structure
- **Need:** flexible model, few parameters - neural networks

Model family	Key idea
Autoregressive models	one variable at a time, given the previous ones
Latent variable models, VAEs	hidden code $\mathbf{z}$, decoded to $\mathbf{x}$
Normalizing flows	invertible map from simple noise to data
GANs	generator against discriminator
Energy-based and score models	unnormalised density, its gradient
Diffusion, flow matching	noise to data in many small steps

Along the way: maximum likelihood, KL divergence, ELBO, evaluation, conditional generation

If time allows: self-supervised learning, inverse modeling

Schedule

- Tue 13:30 - 15:00, Thu 14:00 - 15:30
- CAIRO 3rd floor left meeting room

Teaching

- Lecture: slides and provided refs (papers, book chapters, etc.)
- Exercises: implement models yourself - Jupyter notebook, individual
- Compute: Colab or THWS GPUs

Evaluation

- Individual project: test on real data / reimplement paper / own idea
- Code + short report + presentation
- Report: what matched, what diverged